Student Name: Pham Xuan Hoang
Student ID: 2640009

Information Systems
Labwork 4

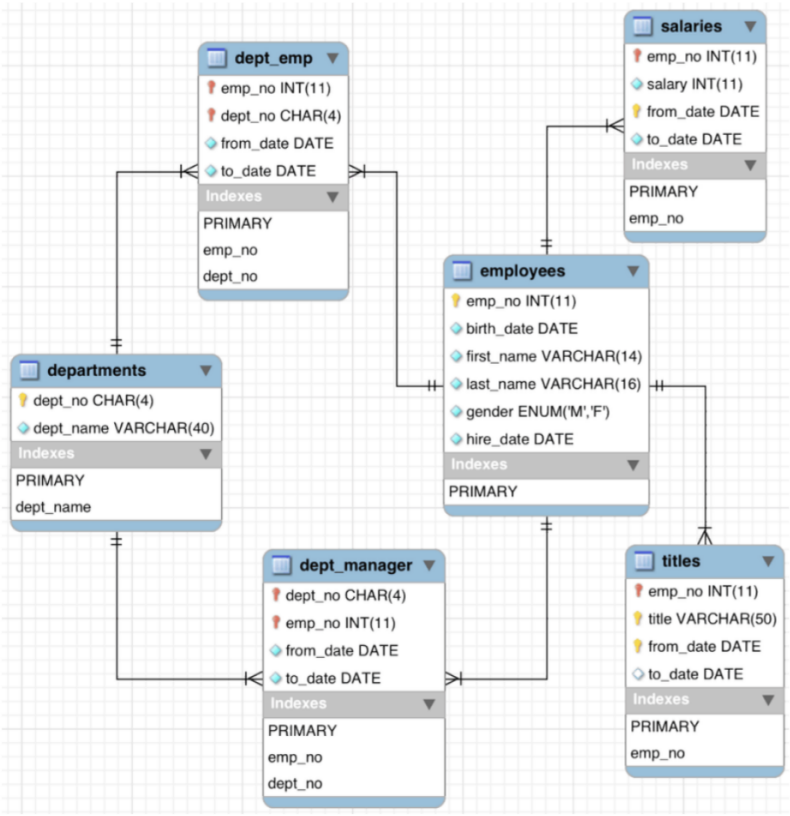


Figure 1: `employees` database schema

Built-in function

Write SQL commands for the following queries:

- What is the average salary of each employee? (output shortened)

```sql
SELECT
    emp_no,
    AVG(salary) AS avg_salary
FROM salaries
GROUP BY emp_no
LIMIT 100;
```

Output:

```
+--------+-------------+
| emp_no | avg_salary  |
+--------+-------------+
```

```
|  10001 |  75388.9412 |
|  10002 |  68854.5000 |
|  10003 |  43030.2857 |
|  10004 |  56512.2500 |
|  10005 |  87275.7692 |
+--------+-------------+
```

- How much was each employee paid in total? (output shortened)

```sql
SELECT
    emp_no,
    SUM(salary) AS total_salary
FROM salaries
GROUP BY emp_no
LIMIT 100;
```

Output:

```
+--------+--------------+
| emp_no | total_salary |
+--------+--------------+
|  10001 |      1281612 |
|  10002 |       413127 |
|  10003 |       301212 |
|  10004 |       904196 |
|  10005 |      1134585 |
+--------+--------------+
```

- Minimum, maximum and total salaries of each department

```sql
SELECT
    de.dept_no,
    MIN(s.salary) AS min_salary,
    MAX(s.salary) AS max_salary,
    SUM(s.salary) AS total_salary
FROM salaries AS s
INNER JOIN dept_emp AS de ON s.emp_no = de.emp_no
GROUP BY de.dept_no;
```

Output:

```
+---------+------------+------------+--------------+
| dept_no | min_salary | max_salary | total_salary |
+---------+------------+------------+--------------+
| d001    |      39127 |     145128 |  13725425266 |
| d002    |      38812 |     142395 |  11650834677 |
| d003    |      38735 |     141953 |   9363811425 |
| d004    |      38623 |     138273 |  41554438942 |
| d005    |      38849 |     144434 |  48179456393 |
| d006    |      38786 |     132103 |  10865203635 |
| d007    |      39169 |     158220 |  40030089342 |
| d008    |      38851 |     130211 |  11969730427 |
| d009    |      38836 |     144866 |  13143639841 |
+---------+------------+------------+--------------+
```

- Which department paid more than 20 billion dollars for their employee

```sql
SELECT
    d.dept_name,
    SUM(s.salary) AS total_salary
FROM salaries AS s
INNER JOIN dept_emp AS de ON s.emp_no = de.emp_no
INNER JOIN departments AS d ON de.dept_no = d.dept_no
GROUP BY de.dept_no
HAVING total_salary > 20000000000;
```

Output:

```
+-------------+--------------+
| dept_name   | total_salary |
+-------------+--------------+
| Production  |  41554438942 |
| Development |  48179456393 |
| Sales       |  40030089342 |
+-------------+--------------+
```

- Total salaries of male employees in each department

```sql
SELECT
    d.dept_name,
    SUM(s.salary) AS total_male_salary
FROM salaries AS s
INNER JOIN dept_emp AS de ON s.emp_no = de.emp_no
INNER JOIN departments AS d ON de.dept_no = d.dept_no
INNER JOIN employees AS e ON s.emp_no = e.emp_no
WHERE e.gender = "M"
GROUP BY de.dept_no;
```

Output:

```
+--------------------+-------------------+
| dept_name          | total_male_salary |
+--------------------+-------------------+
| Marketing          |        8352455367 |
| Finance            |        6940675318 |
| Human Resources    |        5619533959 |
| Production         |       24873889749 |
| Development        |       28961357095 |
| Quality Management |        6472073211 |
| Sales              |       24051537190 |
| Research           |        7174578852 |
| Customer Service   |        7865587983 |
+--------------------+-------------------+
```

- Total salaries of department managers for each department, from highest to lowest

```sql
SELECT
    d.dept_name,
    SUM(s.salary) AS total_salary
FROM salaries AS s
INNER JOIN dept_manager AS dm ON s.emp_no = dm.emp_no
INNER JOIN departments AS d ON dm.dept_no = d.dept_no
GROUP BY dm.dept_no
ORDER BY total_salary DESC;
```

Output:

```
+--------------------+--------------+
| dept_name          | total_salary |
+--------------------+--------------+
| Quality Management |      4162118 |
| Production         |      3374004 |
| Customer Service   |      3187661 |
| Marketing          |      3093009 |
| Sales              |      2915118 |
| Research           |      2558661 |
| Finance            |      2549372 |
| Human Resources    |      2098298 |
| Development        |      2028376 |
+--------------------+--------------+
```

Subquery

Write SQL command for the following queries:

- Who have the same name as the managers of the “Finance” department

```sql
SELECT *
FROM employees AS x
WHERE (
    x.first_name,
    x.last_name
) IN (
    SELECT
        e.first_name,
        e.last_name
    FROM employees AS e
    INNER JOIN dept_manager AS dm ON e.emp_no = dm.emp_no
    INNER JOIN departments AS d ON dm.dept_no = d.dept_no
    WHERE d.dept_name = 'Finance'
) AND x.emp_no NOT IN (
    SELECT dm.emp_no
    FROM dept_manager AS dm
    INNER JOIN departments AS d ON dm.dept_no = d.dept_no
    WHERE d.dept_name = 'Finance'
);
```

Output:

```
No output
```

- Who in the “Production” department were hired after the promotion of the last manager in that department

```sql
SELECT
    e.emp_no,
    e.first_name,
    e.last_name
FROM employees AS e
INNER JOIN dept_emp AS de ON e.emp_no = de.emp_no
INNER JOIN departments AS d ON de.dept_no = d.dept_no
WHERE
    d.dept_name = 'Production'
    AND e.hire_date > (
        SELECT MAX(s_dm.from_date)
        FROM dept_manager AS s_dm
        INNER JOIN departments AS s_d ON s_dm.dept_no = s_d.dept_no
        WHERE s_d.dept_name = 'Production'
    );
```

Output:

```
+--------+----------------+------------------+
| emp_no | first_name     | last_name        |
+--------+----------------+------------------+
|  10024 | Suzette        | Pettey           |
|  10218 | Zhenhua        | Magalhaes        |
|  10267 | Shaunak        | Cullers          |
|  10284 | Masali         | Murrill          |
|  10298 | Dietrich       | DuCasse          |
|  10552 | Kazuhito       | Shiratori        |
|  10684 | Aimee          | Tokunaga         |
+--------+----------------+------------------+
```

- Find the average salary of each department, from highest to lowest

```sql
SELECT
    d.dept_name,
    AVG(x.salary) AS avg_salary
FROM (
    SELECT
        de.dept_no,
        s.salary
    FROM dept_emp AS de
    INNER JOIN
        salaries AS s
        ON
            de.emp_no = s.emp_no
            AND de.from_date <= s.from_date
            AND de.to_date >= s.to_date
) AS x
INNER JOIN departments AS d ON x.dept_no = d.dept_no
GROUP BY d.dept_name
ORDER BY avg_salary DESC;
```

Output:

```
+--------------------+------------+
| dept_name          | avg_salary |
+--------------------+------------+
| Sales              | 80718.3963 |
| Marketing          | 71907.8664 |
| Finance            | 70828.9095 |
| Research           | 59999.0935 |
| Production         | 59700.3958 |
| Development        | 59391.6657 |
| Customer Service   | 57628.9506 |
| Quality Management | 57276.5570 |
| Human Resources    | 55136.3614 |
+--------------------+------------+
```

- Find the average salary for each type of Engineer, from highest to lowest

```sql
SELECT
    x.title,
    AVG(x.salary) AS avg_salary
FROM
    (
        SELECT
            t.title,
            s.salary
        FROM titles AS t
        INNER JOIN
            salaries AS s
            ON
                t.emp_no = s.emp_no
                AND t.from_date <= s.from_date
                AND t.to_date >= s.to_date
        WHERE t.title LIKE '%Engineer'
    ) AS x
GROUP BY x.title
ORDER BY avg_salary DESC;
```

Output:

```
+--------------------+------------+
| title              | avg_salary |
+--------------------+------------+
| Senior Engineer    | 64729.9696 |
| Engineer           | 54859.3469 |
| Assistant Engineer | 54104.0625 |
+--------------------+------------+
```

View

Write SQL command for the following queries:

- Who have the same name as the managers of the "Finance" department

```sql
CREATE OR REPLACE VIEW finance_manager_fullname AS
SELECT
    e.emp_no,
    e.first_name,
    e.last_name
FROM employees AS e
INNER JOIN dept_manager AS dm ON e.emp_no = dm.emp_no
WHERE
    dm.dept_no = (
        SELECT d.dept_no FROM departments AS d
        WHERE d.dept_name = "Finance"
    );
```

Output:

```
No output
```

- Who in the "Production" department were hired after the promotion of the last manager in that department

```sql
CREATE OR REPLACE VIEW production_manager_promotion AS
SELECT MAX(dm.from_date) AS last_promotion_date
FROM dept_manager AS dm
INNER JOIN departments AS d ON dm.dept_no = d.dept_no
WHERE d.dept_name = 'Production';

SELECT
    e.emp_no,
    e.first_name,
    e.last_name
FROM employees AS e
INNER JOIN dept_emp AS de ON e.emp_no = de.emp_no
INNER JOIN departments AS d ON de.dept_no = d.dept_no
WHERE
    d.dept_name = 'Production'
    AND e.hire_date
    > (SELECT pmp.last_promotion_date FROM production_manager_promotion AS pmp);
```

Output:

```
+--------+---------------+------------------+
| emp_no | first_name    | last_name        |
+--------+---------------+------------------+
|  10024 | Suzette       | Pettey           |
|  10218 | Zhenhua       | Magalhaes        |
|  10267 | Shaunak       | Cullers          |
|  10284 | Masali        | Murrill          |
```

```
|  10298 | Dietrich       | DuCasse          |
|  10552 | Kazuhito       | Shiratori        |
|  10684 | Aimee          | Tokunaga         |
+--------+----------------+------------------+
```

- Find the average salary of each department, from highest to lowest

```sql
CREATE OR REPLACE VIEW avg_salary_dept AS
SELECT
    de.dept_no,
    AVG(s.salary) AS avg_salary
FROM salaries AS s
INNER JOIN
    dept_emp AS de
    ON
        s.emp_no = de.emp_no
        AND s.from_date >= de.from_date
        AND s.to_date <= de.to_date
GROUP BY de.dept_no;

SELECT
    d.dept_name,
    asd.avg_salary
FROM avg_salary_dept AS asd
INNER JOIN departments AS d ON asd.dept_no = d.dept_no
ORDER BY asd.avg_salary DESC;
```

Output:

```
+--------------------+------------+
| dept_name          | avg_salary |
+--------------------+------------+
| Sales              | 80718.3963 |
| Marketing          | 71907.8664 |
| Finance            | 70828.9095 |
| Research           | 59999.0935 |
| Production         | 59700.3958 |
| Development        | 59391.6657 |
| Customer Service   | 57628.9506 |
| Quality Management | 57276.5570 |
| Human Resources    | 55136.3614 |
+--------------------+------------+
```

- Find the average salary for each type of Engineer, from highest to lowest

```sql
CREATE OR REPLACE VIEW avg_engineer_salary AS
SELECT
    t.title,
    AVG(s.salary) AS avg_salary
FROM titles AS t
INNER JOIN
    salaries AS s
    ON
        t.emp_no = s.emp_no
        AND t.from_date <= s.from_date
        AND t.to_date >= s.to_date
WHERE t.title LIKE '%Engineer'
GROUP BY t.title;

SELECT
    title,
    avg_salary
FROM avg_engineer_salary
ORDER BY avg_salary DESC;
```

Output:

```
+--------------------+------------+
| title              | avg_salary |
+--------------------+------------+
| Senior Engineer    | 64729.9696 |
| Engineer           | 54859.3469 |
| Assistant Engineer | 54104.0625 |
+--------------------+------------+
```

Review

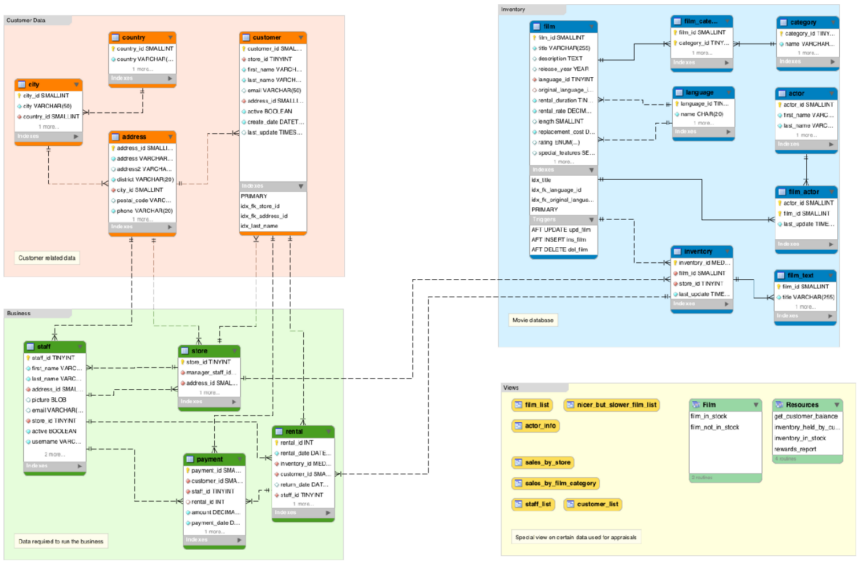


Figure 2: `sakila` database schema

- List names of all the languages in the database (sorted alphabetically)

```sql
SELECT name
FROM language
ORDER BY name;
```

Output:

```
+----------+
| name     |
+----------+
| English  |
| French   |
| German   |
| Italian  |
| Japanese |
| Mandarin |
+----------+
```

- List full names of actors with “GER” in their last name, ordered by their first name

```sql
-- ordered by their first name
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM actor
WHERE last_name LIKE '%GER%'
```

Output:

```
+-------------------+
| full_name         |
+-------------------+
| CAMERON ZELLWEGER |
| JULIA ZELLWEGER   |
| MICHAEL BOLGER    |
| MINNIE ZELLWEGER  |
| VAL BOLGER        |
| VIVIEN BASINGER   |
+-------------------+
```

- Find all the addresses where postal code starts with "57", and return addresses sorted.

```sql
SELECT
    address,
    postal_code
FROM address
WHERE postal_code LIKE '57%'
ORDER BY address;
```

Output:

```
+-----------------------------------+-------------+
| address                           | postal_code |
+-----------------------------------+-------------+
| 1029 Dzerzinsk Manor              | 57519       |
| 1061 Ede Avenue                   | 57810       |
| 1287 Xiangfan Boulevard           | 57844       |
| 1309 Weifang Street               | 57338       |
| 1515 Korla Way                    | 57197       |
| 1697 Kowloon and New Kowloon Loop | 57807       |
| 231 Kaliningrad Place             | 57833       |
| 390 Wroclaw Way                   | 5753        |
| 431 Szekesfehervar Avenue         | 57828       |
| 687 Alessandria Parkway           | 57587       |
| 869 Shikarpur Way                 | 57380       |
+-----------------------------------+-------------+
```

- How many films involve a "DWARF" in their titles?

```sql
SELECT COUNT(*) AS number_of_film_title_involve_dwarf FROM film
```

Output:

```
+-----------------------------------+
| number_of_film_title_involve_dwarf |
+-----------------------------------+
|                                 2 |
+-----------------------------------+
```

- Find full names of actors who played in a film involving "WAR" in title and longer than 2.5 hours, along with the title, run length and release year of the movie. Sorted by the actors last names. (output shortened)

```sql
SELECT
    f.title,
    f.length,
    f.release_year,
    GROUP_CONCAT(
        CONCAT(a.first_name, ' ', a.last_name)
        ORDER BY a.last_name SEPARATOR ', '
    ) AS actors_list
FROM actor AS a
```

```
INNER JOIN film_actor AS fa ON a.actor_id = fa.actor_id
INNER JOIN film AS f ON fa.film_id = f.film_id
WHERE
    fa.film_id IN (
        SELECT fs.film_id
        FROM film AS fs
        WHERE fs.title LIKE '%WAR%' AND fs.length > 150
    )
GROUP BY f.title, f.length, f.release_year
ORDER BY f.title;
```

Output:

```
+--------------------+--------+--------------+------------------------------------------
| title              | length | release_year | actors_list
+--------------------+--------+--------------+------------------------------------------
| GLADIATOR WESTWARD |    173 |         2006 | ADAM GRANT, KIRK JOVOVICH, JULIA MCQUEEN, J
| LEATHERNECKS DWARFS |    153 |         2006 | KARL BERRY, SUSAN DAVIS, SPENCER DEPP, BOB
| WARDROBE PHANTOM   |    178 |         2006 | ANGELINA ASTAIRE, RIVER DEAN, GREGORY GOODI
+--------------------+--------+--------------+------------------------------------------
```

- Find all the film categories in which there are between 55 and 65 films. Return the names of these categories and the number of films per category, sorted by the number of films descending.

```
SELECT
    c.name,
    COUNT(fc.film_id) AS total_film
FROM category AS c
INNER JOIN film_category AS fc ON c.category_id = fc.category_id
GROUP BY c.category_id
HAVING total_film >= 55 AND total_film <= 65
ORDER BY total_film DESC;
```

Output:

```
+----------+------------+
| name     | total_film |
+----------+------------+
| Action   |         64 |
| New      |         63 |
| Drama    |         62 |
| Sci-Fi   |         61 |
| Games    |         61 |
| Children |         60 |
| Comedy   |         58 |
| Classics |         57 |
| Travel   |         57 |
| Horror   |         56 |
+----------+------------+
```

- In how many film categories is the average difference between the film replacement cost and the rental rate larger than 17?

```
WITH rent_rate AS (
    SELECT
        fc.category_id,
        AVG(f.rental_rate) AS avg_rental_rate
    FROM film AS f
    INNER JOIN film_category AS fc ON f.film_id = fc.film_id
    GROUP BY fc.category_id
)

SELECT
    rep_cost.category_id,
```

```
    rep_cost.avg_replacement_cost - rent_rate.avg_rental_rate AS avg_diff
FROM (
    SELECT
        fc.category_id,
        AVG(f.replacement_cost) AS avg_replacement_cost
    FROM film AS f
    INNER JOIN film_category AS fc ON f.film_id = fc.film_id
    GROUP BY fc.category_id
) AS rep_cost
INNER JOIN rent_rate ON rep_cost.category_id = rent_rate.category_id
HAVING avg_diff > 17;
```

Alternate approach using `CREATE VIEW`

```
CREATE OR REPLACE VIEW avg_rental_rate_by_category AS
SELECT
    fc.category_id,
    AVG(f.rental_rate) AS avg_rental_rate
FROM film AS f
INNER JOIN film_category AS fc ON f.film_id = fc.film_id
GROUP BY fc.category_id;

CREATE OR REPLACE VIEW avg_replacement_cost_by_category AS
SELECT
    fc.category_id,
    AVG(f.replacement_cost) AS avg_replacement_cost
FROM film AS f
INNER JOIN film_category AS fc ON f.film_id = fc.film_id
GROUP BY fc.category_id;

SELECT
    avg_cost.category_id,
    avg_cost.avg_replacement_cost - avg_rent.avg_rental_rate AS avg_diff
FROM avg_rental_rate_by_category AS avg_rent
INNER JOIN
    avg_replacement_cost_by_category AS avg_cost
    ON avg_rent.category_id = avg_cost.category_id
HAVING avg_diff > 17;
```

Output:

```
+-------------+-----------+
| category_id | avg_diff  |
+-------------+-----------+
|           1 | 18.265625 |
|           2 | 17.318182 |
|           3 | 17.166667 |
|           4 | 18.263158 |
|           7 | 18.064516 |
|          10 | 17.032787 |
|          14 | 17.934426 |
|          15 | 17.270270 |
+-------------+-----------+
```

- Find the address district(s) name(s) such that the minimal postal code in the district(s) is maximal over all the districts. Make sure your query ignores empty postal codes and district names.

```
SELECT district FROM address
WHERE TRIM(district) <> '' AND TRIM(postal_code) <> ''
GROUP BY district
HAVING
    MIN(postal_code) = (
```

```
        SELECT MAX(min_postal_code.min_code) AS max_min_postal_code
        FROM (
            SELECT MIN(postal_code) AS min_code
            FROM address
            WHERE TRIM(district) <> '' AND TRIM(postal_code) <> ''
            GROUP BY district
        ) AS min_postal_code
    );
```

Output:

```
+----------+
| district |
+----------+
| Cordoba  |
+----------+
```

- Find the names (first and last) of all the actors and customers whose first name is the same as the first name of the actor with ID 101 (exclude the actor with ID 101).

```
SELECT *
FROM (
    (SELECT
        actor_id AS combined_id,
        first_name,
        last_name
    FROM actor)
    UNION
    (SELECT
        customer_id AS combined_id,
        first_name,
        last_name
    FROM customer)
) AS name_table
WHERE name_table.last_name = (
    SELECT a.last_name FROM actor AS a
    WHERE a.actor_id = 101
) AND name_table.combined_id <> 101;
```

Output:

```
+-------------+------------+-----------+
| combined_id | first_name | last_name |
+-------------+------------+-----------+
|           4 | JENNIFER   | DAVIS     |
|         110 | SUSAN      | DAVIS     |
|           6 | JENNIFER   | DAVIS     |
+-------------+------------+-----------+
```